

Course: Microprocessors	Section: 3-6
Name/s: Carl Errol G. Cabali	Date Performed: June 1, 2026
	Date Submitted: June 10, 2026
	Instructor: Engr. Rufo I. Marasigan Jr.
Laboratory Activity No.6	
Ultrasonic Sensor	
1. Objective(s):	
This activity covers the operation of ultrasonic sensors, including their components and fundamental functions in distance measurement.	
2. Intended Learning Outcomes (ILOs):	
At the end of the activity, the student shall be able to: ● Use ultrasonic sensor as input to the Arduino ● Use an LCD to visually display the Arduino output derived from the ultrasonic sensor ● Manipulate C/C++ code to implement the required output sequence	
3. Materials	
● Arduino Uno R3 ● HC-SR04 Ultrasonic Distance Sensor ● Breadboard and jumper wires ● Pushbutton (for measurement trigger) ● Piezo Buzzer	
4. Discussion	
Ultrasonic Sensor Ultrasonic sensors or ultrasonic transducers generate high-frequency sound waves, at a frequency around 20 kHz, depending on the purpose. The ultrasonic sensor converts electrical signals (high-frequency sound waves) into high-frequency vibration. An ultrasonic sensor converts electrical energy into mechanical energy. This transduction is an application of piezoelectricity or the piezoelectric effect. The piezoelectric effect was discovered in 1880 by the brothers Pierre and Jacques Curie, so it is also called the Pierre effect.	

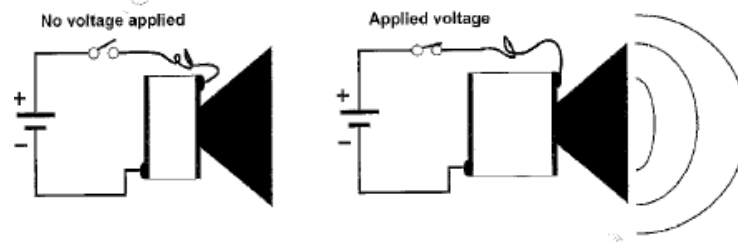


Figure 6-1 Ultrasonic transmitter

Ultrasonic Transmitter

The operation of an ultrasonic transmitter is based on the reverse piezoelectric effect. When a pulse signal is applied to piezoelectric material, the pulse stretches and compresses the piezoelectric material according to the pulse frequency. The piezoelectric material generates high-frequency sound waves which propagate in air, as shown in Figure 6-1.

Ultrasonic Receiver

The operation of an ultrasonic receiver is based on the direct piezoelectric effect. When the piezoelectric material receives high-frequency sound waves (mechanical force), it generates voltages varying with the frequency of the sound wave, as shown in Figure 6-2.

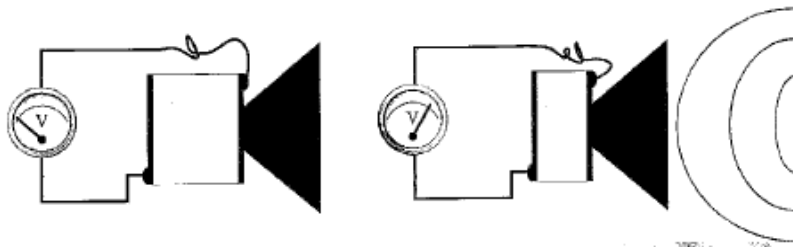


Figure 6-2 Ultrasonic Receiver

Simple Ultrasonic Range Finder

If an ultrasonic transmitter and an ultrasonic receiver are placed face to face, as shown in Figure 6-3, the transmitter generates ultrasonic waves propagating through air that are received by the receiver. The speed of sound refers to the speed of sound waves in air, which is about 346 m/s at 25°C. If the propagation time t of the ultrasonic waves from transmitter to receiver is measured, the distance between them can be calculated using the formula: distance = 346 m/s $\times$ t .

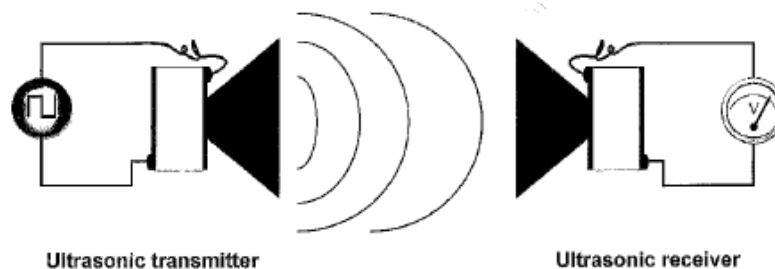


Figure 6-3 Ultrasonic Range Measurement

Practical Ultrasonic Range Finder

A reflective ultrasonic range finder is made by arranging the transmitter and receiver side by side, as shown in Figure 6-4. The operating principle of a reflective ultrasonic range finder is based on an ultrasonic wave that is reflected when hitting an obstacle. Once the ultrasonic receiver captures the reflected wave, the propagation time t from transmitter to receiver is measured, and the distance between the finder and the obstacle is calculated by the equation: distance = $(346 \text{ m/s} \times t) / 2$.

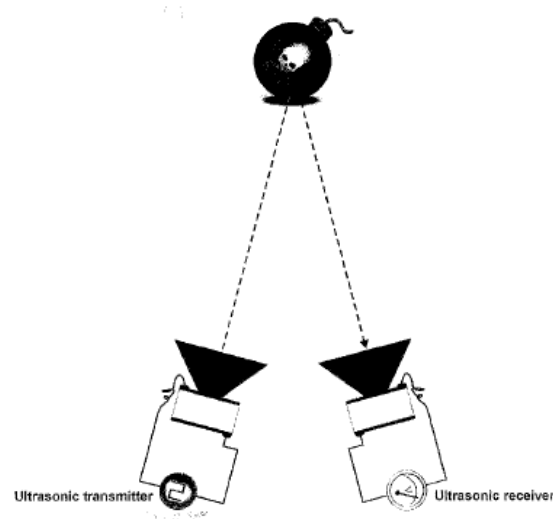


Figure 6-4 Reflective ultrasonic Range Finder.

Ultrasonic Module

In the simulation environment, we use the HC-SR04 ultrasonic ranging module. It is a reflective sensor featuring a transmitter (labeled T) and a receiver (labeled R) used to detect obstacles and calculate distance.



Figure 6-4 HC-SR04 Ultrasonic Range Finder Module

The HC-SR04 ultrasonic sensor has 4 pins. The pin function is shown in Table 6-1.

Table 6-1

Pin No.	Pin name	Connection
1	VCC	Power supply, +5V
2	Trig	Transmit signal
3	Echo	Receive Signal
4	GND	Ground

The VCC and GND pins must be connected to the Arduino's 5V and GND rails. The Trig and Echo pins are connected to digital I/O pins for signal processing.

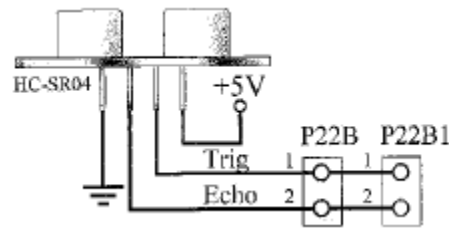


Figure 6-5 Connections of HC-SR04 Ultrasonic Sensor

Ultrasonic Distance Measurement

When a positive trigger pulse is applied to the Trig pin of the HC-SR04 ultrasonic sensor, the transmitter produces ultrasonic waves and the voltage level on the Echo pin becomes HIGH. Once the receiver receives the reflected sound wave, the Echo pin returns to LOW. In other words, the width of the Echo pulse represents the time interval between sending the signal and receiving the echo, which is used to determine the distance to a target.

The distance traveled by the sound wave is equal to twice the distance between the HC-SR04 and the target. If the measured pulse width of the Echo signal is t μ s, the distance d is calculated as:

$$d = (346 \text{ m/s} \times t) / 2 = 173 \times 10^{-4} \text{ cm} \times t = 0.0173 \times t \text{ (cm)}$$

pulseIn() function

The *pulseIn()* function reads a pulse (either HIGH or LOW) on a pin. For example, if value is HIGH, **pulseIn()**. Waits for the pin to go HIGH, starts timing, then waits for the pin to go LOW and stops timing. This function returns the length of the pulse in microseconds or 0 if no complete pulse was received within timeout.

Syntax:

Width = pulseIn (pin, type, timeout);

5. Procedures

In this procedure, you will create a simple ultrasonic range finder to measure the distance to an object using the HC-SR04 sensor and an Arduino Uno. The system will take continuous distance measurements and display the results in centimeters on the Serial Monitor.

1. Open Tinkercad and create a new Circuit design.
2. Place an Arduino Uno and an HC-SR04 ultrasonic distance sensor on the workspace.
3. Connect the HC-SR04 VCC pin to the Arduino 5V pin and the GND pin to the Arduino GND pin.
4. Connect the HC-SR04 Trig pin to Arduino digital pin 9.
5. Connect the HC-SR04 Echo pin to Arduino digital pin 10.
6. Verify all wiring connections to ensure they are secure and correct.

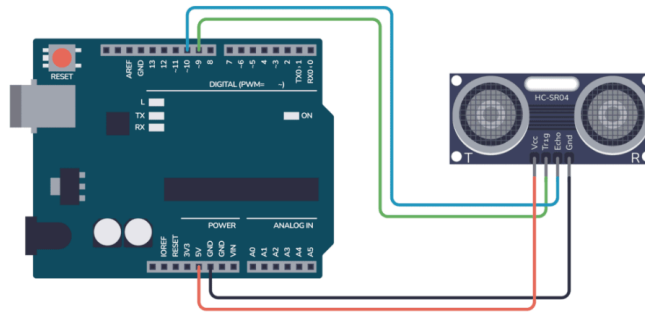


Figure 6-6 Wiring diagram of ultrasonic range sensor

7. Open the 'Code' panel, select 'Text' mode, and write your Arduino code to trigger the sensor and read the echo pulse.

```

1 // HC-SR04 Ultrasonic Sensor with Arduino Uno
2
3 const int trigPin = 9;
4 const int echoPin = 10;
5
6 long duration;
7 float distance;
8
9 void setup() {
10   Serial.begin(9600);
11
12   pinMode(trigPin, OUTPUT);
13   pinMode(echoPin, INPUT);
14 }
15
16 void loop() {
17
18   // Clear trigPin
19   digitalWrite(trigPin, LOW);
20   delayMicroseconds(2);
21
22   // Send 10 microsecond pulse
23   digitalWrite(trigPin, HIGH);
24   delayMicroseconds(10);
25   digitalWrite(trigPin, LOW);
26
27   // Read echo time
28   duration = pulseIn(echoPin, HIGH);
29
30   // Calculate distance in centimeters
31   distance = duration * 0.034 / 2;
32
33   // Display distance

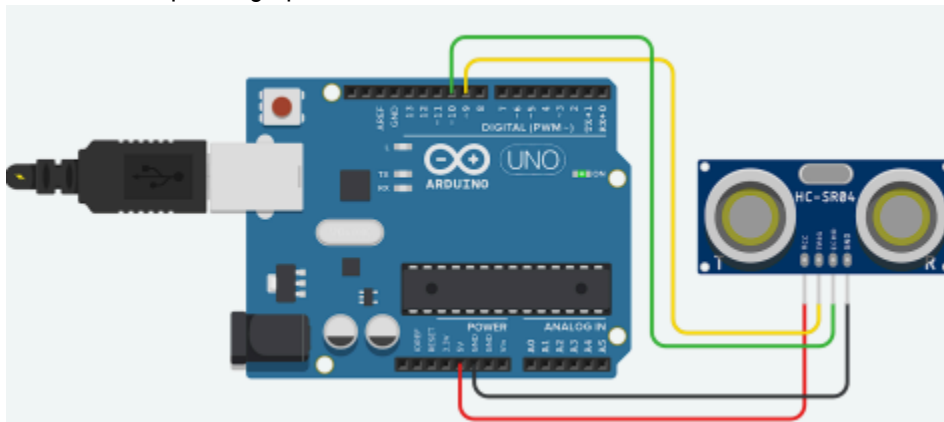
```

```

34 Serial.print("Distance: ");
35 Serial.print(distance);
36 Serial.println(" cm");
37
38 delay(500);
39 }

```

8. Upload the code to the Arduino Uno.
9. Click 'Start Simulation' and open the Serial Monitor to view the continuous distance data.
10. Interact with the ultrasonic sensor in the simulation to change the distance and observe the corresponding updates on the Serial Monitor.



Serial Monitor

```

Distance: 112.08 cm
Distance: 112.08 cm
Distance: 112.08 cm
Distance: 112.08 cm
Distance: 111.89 cm
Distance: 112.08 cm
Distance: 112.08 cm

```

Serial Monitor

```

Distance: 112.08 cm
Distance: 112.08 cm
Distance: 112.08 cm
Distance: 111.86 cm
Distance: 112.05 cm
Distance: 112.08 cm
Distance: 112.08 cm

```

6. Supplemental Activity

Modify the previous ultrasonic sensor simulation by integrating additional output and notification components to create a more interactive and intelligent monitoring system.

Objectives

Enhance the existing Arduino ultrasonic distance monitoring system by implementing the following features:

- Integrate an I2C LCD to display:
 - Current distance reading
 - Average distance after 10 readings
- Program the system to:
 - Take one distance reading every second
 - Collect readings continuously for 10 seconds
 - Compute and display the average distance
- Add an LED Bar Graph (or multiple LEDs) to visually indicate distance levels.
- Add a buzzer notification system that activates when the measured distance reaches a critical threshold.
- Add a push button control:
 - Pressing the button starts the measurement process.
 - The system waits for another button press before restarting the measurement cycle.

Suggested Threshold Levels

- **SAFE CONDITION**
 - Distance greater than 20 cm
 - Green LEDs ON
 - Buzzer OFF
- **WARNING CONDITION**
 - Distance between 10 cm and 20 cm
 - Yellow LEDs ON
 - Buzzer OFF
- **DANGER CONDITION**
 - Distance less than 10 cm

- Red LEDs ON
- Buzzer ON

Required Components

- Arduino Uno
- HC-SR04 Ultrasonic Sensor
- I2C LCD (16x2)
- LED Bar Graph or 5 LEDs
- Piezo Buzzer
- Push Button
- Breadboard
- Jumper Wires

Expected Output

The completed system should be capable of:

- Displaying real-time distance readings on the LCD.
- Computing and displaying the average distance after 10 readings.
- Using LEDs to visually represent distance levels.
- Triggering a buzzer alarm when the object is too close.
- Starting the measurement process only when the push button is pressed.

Guide Questions

1. What is the purpose of averaging multiple sensor readings?
 - The accuracy of the distance measurement is increased by averaging several sensor readings and is steady. Sometimes noise, air movement, or minor mistakes might cause a single reading to be inaccurate. We obtain a more accurate result by averaging ten readings.
2. How does the LED bar graph help visualize sensor data?
 - The LED bar graph provides a quick and intuitive way to interpret distance measurements. Depending on how close an object is, different colored LEDs illuminate to indicate its status: green for safe conditions (above 20 cm), yellow for caution (10 to 20 cm), and red

for danger (below 10 cm). This visual system allows users to instantly understand the situation without needing to read numerical values.

3. Why is threshold monitoring important in embedded systems?

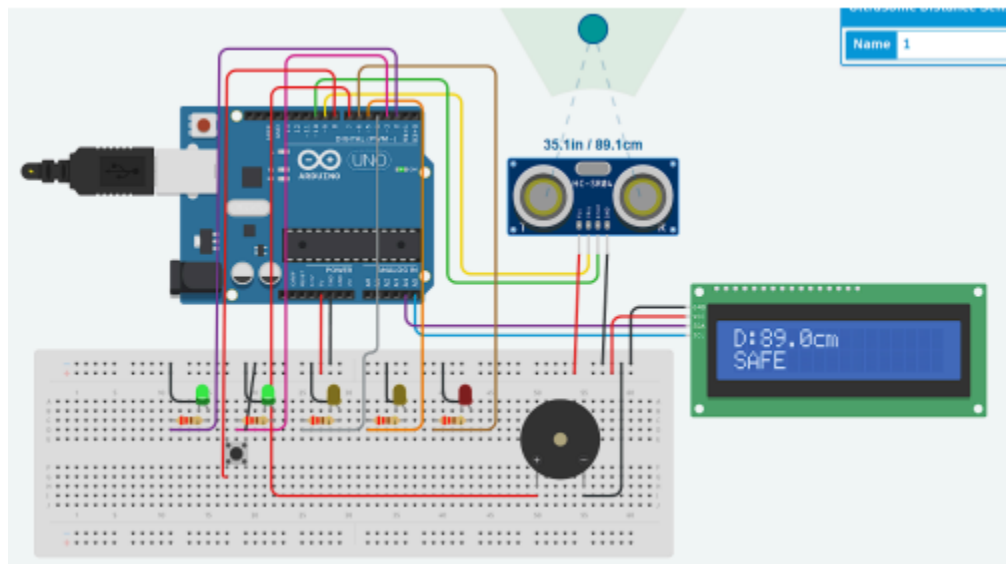
- Threshold monitoring plays an important role in embedded systems because it enables automatic system responses. When the measured distance reaches an unsafe or critical level, the system can immediately trigger actions such as turning on lights, activating an alarm, or performing other predefined responses. This improves both the intelligence and safety of the system.

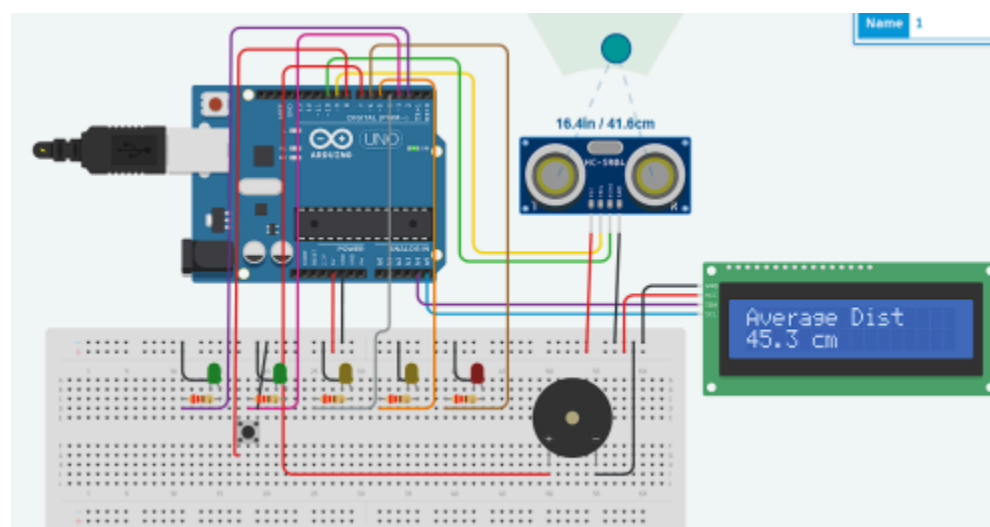
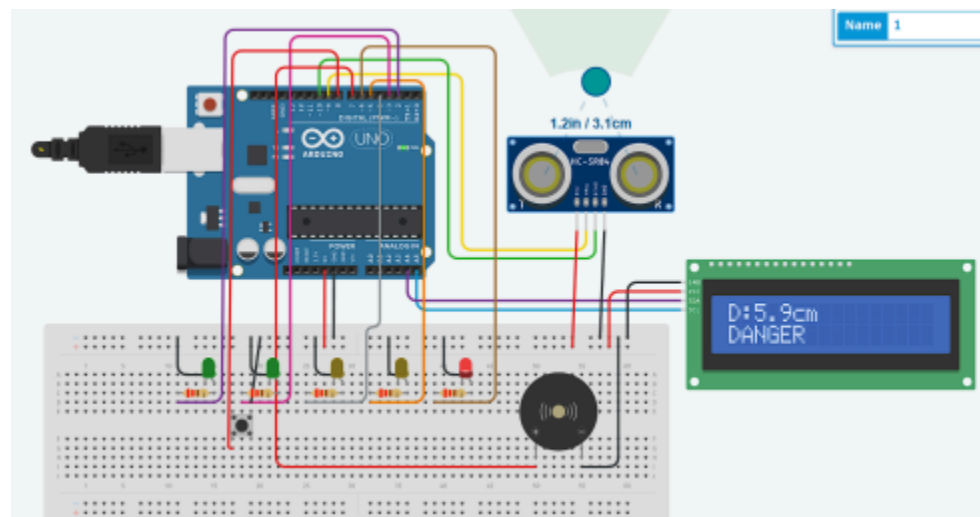
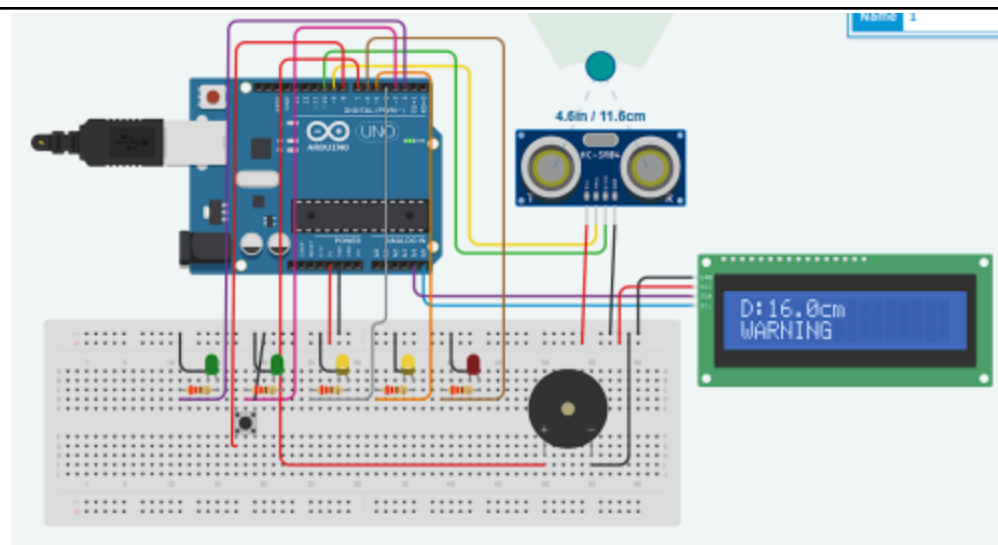
4. Give at least two real-world applications of ultrasonic distance monitoring systems?

- Ultrasonic distance systems are used in real life for car parking sensors to help drivers avoid hitting things when reversing. They are also used in robots and drones to detect and avoid obstacles while moving.

5. What are the advantages of using an I2C LCD compared to a standard LCD connection?

- An I2C LCD is better than a regular LCD because it only needs 2 wires to connect to the Arduino. This saves many pins on the Arduino, makes wiring simpler, and leaves more pins free for other parts like LEDs or the buzzer.





7. Conclusion:

What conclusion can you make based on the activity?

This lab exercise provided a thorough grasp of ultrasonic sensors, from their piezoelectric working principle to its useful Arduino implementation for precise distance measuring. The setup was expanded into a more comprehensive monitoring system that offers visual, numerical, and auditory input across the basic and supplemental duties. All things considered, it reinforced fundamental embedded systems principles and assisted in preparing for more sophisticated IoT and automation applications.

8. Assessment (Rubric for Laboratory Performance):

Intended Learning Outcomes	Unsatisfactory 1	Satisfactory 2	Exemplary 3	Score
Use ultrasonic sensor as input to the Arduino	Unable to use ultrasonic sensor as input to the Arduino	Able to use ultrasonic sensor as input to the Arduino but with errors	Able to use ultrasonic sensor as input to the Arduino without errors	
Implement data collection and average calculation	Unable to collect readings or compute the average correctly	Able to collect readings and compute average but with timing or calculation errors	Correctly collects readings every second for 10 seconds and accurately computes the average	
Use LCD as visual display for the output of the Arduino in the output of the ultrasonic sensor	Unable to use LCD as visual display for the output of the Arduino in the output of the ultrasonic sensor	Able to use LCD as visual display for the output of the Arduino in the output of the ultrasonic sensor but with errors	Able to use LCD as visual display for the output of the Arduino in the output of the ultrasonic sensor without errors	
Manipulate C/C++ code to implement the required output sequence	Unable to manipulate C/C++ code to implement the required output sequence	Able to manipulate C/C++ code to implement the required output sequence but with errors	Able to manipulate C/C++ code to implement the required output sequence without errors	
Other comments/observation:	Total Score			
	RATING = $\frac{\text{total score}}{9} \times 100\%$			

Evaluated by:

Date:

Printed Name and Signature of Faculty Member